Godhard Curve Integration
Non-Linear Dynamics & Critical Transitions in Multi-Agent Systems

---

Godhard Curve Foundation

```python
"""
godhard_curve_dynamics.py
Integration of Godhard curve (non-linear response, hysteresis, catastrophic transitions)
into individual and multi-agent support systems
"""

import numpy as np
from scipy.integrate import odeint
from scipy.optimize import fsolve, brentq
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Tuple, Callable
from enum import Enum
import matplotlib.pyplot as plt

# ============================================
# GODHARD CURVE MATHEMATICS
# ============================================

class GodhardCurve:
    """
    The Godhard Curve (also known as S-curve, sigmoid response, or catastrophe surface)
    represents non-linear response to stress with:

    1. **Gradual phase**: Small inputs → small outputs
    2. **Critical zone**: Small inputs → LARGE outputs (sensitivity peak)
    3. **Plateau phase**: Large inputs → saturated output
    4. **Hysteresis**: Different paths up vs down (history dependence)
    5. **Catastrophic jumps**: Discontinuous state changes at critical points

    Mathematical form:
    dx/dt = -dV/dx where V(x,p) is a potential with multiple minima

    Canonical form (cusp catastrophe):
    V(x,p,q) = x⁴/4 - px²/2 - qx

    Where:
    - x: system state (e.g., strain, distress)
    - p: "control parameter" (e.g., pressure, load)
    - q: "bifurcation parameter" (e.g., margin, resources)
    """

    def __init__(self):
        # Curve shape parameters
        self.steepness = 4.0  # How sharp the S-curve
        self.midpoint = 5.0   # Where transition occurs
        self.min_value = 0.0
        self.max_value = 10.0
```

```
        # Hysteresis parameters
        self.hysteresis_width = 1.5  # Width of hysteresis loop
        self.upper_threshold = 6.5   # Upper jump point (going up)
        self.lower_threshold = 3.5   # Lower jump point (going down)

        # Catastrophe theory parameters
        self.cusp_a = 1.0  # Controls cusp sharpness
        self.cusp_b = 1.0  # Controls bifurcation

        # History tracking for hysteresis
        self.current_branch = "lower"  # "lower" or "upper"
        self.history: List[Tuple[float, float]] = []

    def potential_function(self, x: float, pressure: float, margin: float) -> float:
        """
        Potential function V(x, p, m)
        System seeks to minimize this potential

        V(x) = x⁴/4 - (margin/10)·x²/2 - (pressure/10)·x

        Multiple minima create bistability and catastrophic jumps
        """
        normalized_pressure = pressure / 10.0
        normalized_margin = margin / 10.0

        return (
            0.25 * x**4
            - 0.5 * normalized_margin * x**2
            - normalized_pressure * x
        )

    def force_function(self, x: float, pressure: float, margin: float) -> float:
        """
        Force F(x) = -dV/dx
        Zero crossings are equilibrium points
        """
        normalized_pressure = pressure / 10.0
        normalized_margin = margin / 10.0

        return (
            -x**3
            + normalized_margin * x
            + normalized_pressure
        )

    def find_equilibria(self, pressure: float, margin: float) -> List[float]:
        """
        Find all equilibrium points (stable and unstable)
        These are solutions to F(x) = 0
        """
        equilibria = []

        # Search for roots in reasonable range
        test_points = np.linspace(-5, 15, 100)

        for i in range(len(test_points) - 1):
            try:
```

```
            # Check for sign change (root exists)
            f1 = self.force_function(test_points[i], pressure, margin)
            f2 = self.force_function(test_points[i+1], pressure, margin)

            if f1 * f2 < 0:
                root = brentq(
                    lambda x: self.force_function(x, pressure, margin),
                    test_points[i],
                    test_points[i+1]
                )

                # Check if already found (avoid duplicates)
                if not any(abs(root - eq) < 0.01 for eq in equilibria):
                    equilibria.append(root)
        except:
            continue

    return sorted(equilibria)

def classify_equilibrium_stability(
    self,
    x: float,
    pressure: float,
    margin: float
) -> str:
    """
    Classify equilibrium as stable or unstable
    Stable if d²V/dx² > 0 (local minimum)
    Unstable if d²V/dx² < 0 (local maximum)
    """
    normalized_margin = margin / 10.0

    # Second derivative of potential
    d2V = 3 * x**2 - normalized_margin

    if d2V > 0:
        return "stable"
    elif d2V < 0:
        return "unstable"
    else:
        return "marginal"

def response_with_hysteresis(
    self,
    pressure: float,
    margin: float,
    previous_state: float
) -> Tuple[float, bool]:
    """
    Compute response with hysteresis
    Returns: (new_state, jumped)

    Key insight: SAME input can give DIFFERENT output depending on history
    """
    equilibria = self.find_equilibria(pressure, margin)

    if len(equilibria) == 1:
```

```
            # Single equilibrium - no hysteresis
            return equilibria[0], False

        elif len(equilibria) == 3:
            # Three equilibria: lower stable, middle unstable, upper stable
            # This is where hysteresis happens!
            lower_stable = equilibria[0]
            middle_unstable = equilibria[1]
            upper_stable = equilibria[2]

            # Determine which branch we're on based on history
            if self.current_branch == "lower":
                # On lower branch - check if we should jump to upper
                if previous_state > middle_unstable:
                    # Jump up!
                    self.current_branch = "upper"
                    return upper_stable, True
                else:
                    return lower_stable, False

            else:  # current_branch == "upper"
                # On upper branch - check if we should jump to lower
                if previous_state < middle_unstable:
                    # Jump down!
                    self.current_branch = "lower"
                    return lower_stable, True
                else:
                    return upper_stable, False

        else:
            # Unusual case - default to closest equilibrium
            stable_equilibria = [
                eq for eq in equilibria
                if self.classify_equilibrium_stability(eq, pressure, margin) == "stable"
            ]

            if stable_equilibria:
                closest = min(stable_equilibria, key=lambda eq: abs(eq - previous_state))
                return closest, False
            else:
                return equilibria[0], False

    def compute_catastrophe_manifold(
        self,
        pressure_range: np.ndarray,
        margin_range: np.ndarray
    ) -> Dict[str, np.ndarray]:
        """
        Compute the full catastrophe manifold showing all equilibria
        as function of control parameters

        Returns fold lines where catastrophic jumps occur
        """
        P, M = np.meshgrid(pressure_range, margin_range)

        # Store equilibria surfaces
        lower_surface = np.zeros_like(P)
```

```
    upper_surface = np.zeros_like(P)
    middle_surface = np.zeros_like(P)

    n_equilibria = np.zeros_like(P)

    for i in range(P.shape[0]):
        for j in range(P.shape[1]):
            p = P[i, j]
            m = M[i, j]

            equilibria = self.find_equilibria(p, m)
            n_equilibria[i, j] = len(equilibria)

            if len(equilibria) >= 1:
                lower_surface[i, j] = equilibria[0]
            if len(equilibria) >= 2:
                middle_surface[i, j] = equilibria[1]
            if len(equilibria) >= 3:
                upper_surface[i, j] = equilibria[2]

    return {
        'pressure': P,
        'margin': M,
        'lower_surface': lower_surface,
        'middle_surface': middle_surface,
        'upper_surface': upper_surface,
        'n_equilibria': n_equilibria
    }

def compute_resilience(
    self,
    current_state: float,
    pressure: float,
    margin: float
) -> float:
    """
    Compute resilience: how far from catastrophic transition

    Resilience = depth of potential well
    Higher resilience = more perturbation needed to escape basin
    """
    # Find current equilibrium basin
    equilibria = self.find_equilibria(pressure, margin)

    # Find which equilibrium we're near
    current_equilibrium = min(equilibria, key=lambda eq: abs(eq - current_state))

    # Compute potential at current equilibrium
    V_current = self.potential_function(current_equilibrium, pressure, margin)

    # Find saddle points (unstable equilibria) - these are escape routes
    unstable_equilibria = [
        eq for eq in equilibria
        if self.classify_equilibrium_stability(eq, pressure, margin) == "unstable"
    ]

    if not unstable_equilibria:
```

```
            # No escape route - high resilience
            return 10.0

        # Resilience = barrier height to nearest saddle
        barrier_heights = [
            self.potential_function(eq, pressure, margin) - V_current
            for eq in unstable_equilibria
        ]

        min_barrier = min(barrier_heights)

        # Normalize to [0, 10] scale
        resilience = np.clip(min_barrier * 5, 0, 10)

        return resilience


# ============================================
#  INTEGRATION WITH INDIVIDUAL DYNAMICS
# ============================================

@dataclass
class GodhardAwareSystemState:
    """
    System state that tracks position on Godhard curve
    """
    base_state: 'SystemState'

    # Godhard curve position
    curve_position: float = 5.0  # Where on S-curve [0-10]

    # Critical transition tracking
    in_critical_zone: bool = False
    distance_to_fold: float = 5.0  # How close to catastrophic jump
    resilience: float = 5.0  # Depth of stability basin

    # Hysteresis tracking
    on_upper_branch: bool = False
    branch_history: List[str] = field(default_factory=list)

    # Sensitivity
    sensitivity: float = 1.0  # dOutput/dInput (slope of curve)

    def update_godhard_metrics(
        self,
        godhard_curve: GodhardCurve,
        pressure: float,
        margin: float
    ):
        """Update all Godhard-related metrics"""
        # Find position on curve
        equilibria = godhard_curve.find_equilibria(pressure, margin)

        if equilibria:
            current_eq = min(equilibria, key=lambda eq: abs(eq - self.curve_position))
            self.curve_position = current_eq
```

```
        # Determine if in critical zone
        self.in_critical_zone = len(equilibria) >= 3

        # Compute resilience
        self.resilience = godhard_curve.compute_resilience(
            self.curve_position, pressure, margin
        )

        # Compute sensitivity (numerical derivative)
        epsilon = 0.1
        eq_plus = godhard_curve.find_equilibria(pressure + epsilon, margin)
        if eq_plus:
            closest_plus = min(eq_plus, key=lambda eq: abs(eq - self.curve_position))
            self.sensitivity = abs(closest_plus - self.curve_position) / epsilon

        # Distance to fold (catastrophic transition)
        if len(equilibria) >= 3:
            middle_unstable = equilibria[1]
            self.distance_to_fold = abs(self.curve_position - middle_unstable)
        else:
            self.distance_to_fold = 10.0


class GodhardDynamicsEngine:
    """
    Dynamics engine that respects Godhard curve non-linearities
    """

    def __init__(self):
        self.godhard_curve = GodhardCurve()
        self.warning_threshold = 2.0  # Distance to fold that triggers warning

    def evolve_with_godhard(
        self,
        current_state: GodhardAwareSystemState,
        control_input: 'ControlInput',
        dt: float = 1.0
    ) -> Tuple[GodhardAwareSystemState, List[str]]:
        """
        Evolve system respecting Godhard curve dynamics
        Returns new state + warnings about critical transitions
        """
        warnings = []

        # Extract base parameters
        base = current_state.base_state
        pressure = base.pressure
        margin = base.margin

        # Apply control (affects pressure and margin)
        new_pressure = pressure - control_input.pressure_relief + control_input.external_load_change
        new_margin = margin + control_input.capacity_building - control_input.margin_erosion

        new_pressure = np.clip(new_pressure, 0, 10)
        new_margin = np.clip(new_margin, 0, 10)

        # Check for catastrophic transition
```

```
        new_position, jumped = self.godhard_curve.response_with_hysteresis(
            new_pressure,
            new_margin,
            current_state.curve_position
        )

        if jumped:
            if new_position > current_state.curve_position:
                warnings.append(" CATASTROPHIC JUMP UP: System jumped to high-strain state!")
                warnings.append("   This is a non-linear phase transition - small changes had large effect")
            else:
                warnings.append("   CATASTROPHIC JUMP DOWN: System dropped to low-strain state!")
                warnings.append("   Recovery transition occurred - now in healthier basin")

        # Check if entering critical zone
        equilibria = self.godhard_curve.find_equilibria(new_pressure, new_margin)
        in_critical_zone = len(equilibria) >= 3

        if in_critical_zone and not current_state.in_critical_zone:
            warnings.append("    ENTERING CRITICAL ZONE: Small changes may cause large effects!")
            warnings.append("   System now has multiple stable states (bistability)")

        # Check distance to fold
        if in_critical_zone:
            middle_unstable = equilibria[1]
            distance_to_fold = abs(new_position - middle_unstable)

            if distance_to_fold < self.warning_threshold:
                warnings.append(f"   NEAR CATASTROPHIC FOLD: Only {distance_to_fold:.2f} units from
discontinuous jump")
                warnings.append("   Extreme caution - very high sensitivity to perturbations")

        # Compute resilience
        resilience = self.godhard_curve.compute_resilience(new_position, new_pressure, new_margin)

        if resilience < 3.0:
            warnings.append(f" LOW RESILIENCE: Shallow stability basin (resilience={resilience:.1f})")
            warnings.append("   System can be easily knocked into different state")

        # Create new state
        new_base_state = SystemState(
            timestamp=base.timestamp + timedelta(hours=dt),
            pressure=new_pressure,
            margin=new_margin,
            temperature=base.temperature,  # Simplified
            drift=base.drift,
            escalation_risk=1.0 if in_critical_zone else 0.3
        )

        new_godhard_state = GodhardAwareSystemState(
            base_state=new_base_state,
            curve_position=new_position,
            in_critical_zone=in_critical_zone,
            distance_to_fold=distance_to_fold if in_critical_zone else 10.0,
            resilience=resilience,
            on_upper_branch=self.godhard_curve.current_branch == "upper"
        )
```

```
        new_godhard_state.update_godhard_metrics(
            self.godhard_curve,
            new_pressure,
            new_margin
        )

        return new_godhard_state, warnings


# ============================================
#  GODHARD EFFECTS IN MULTI-AGENT SYSTEMS
# ============================================

class MultiAgentGodhardDynamics:
    """
    Godhard curve effects in multi-agent systems

    Key phenomena:
    1. **Contagious catastrophes**: One agent's jump triggers others
    2. **Collective hysteresis**: Group harder to move down than up
    3. **Critical slowing down**: System slows near transition
    4. **Early warning signals**: Variance increases before transition
    """

    def __init__(self, network: 'SocialNetwork'):
        self.network = network
        self.godhard_curves = {
            agent_id: GodhardCurve()
            for agent_id in network.agents.keys()
        }

        # Coupling strength for catastrophe contagion
        self.contagion_strength = 0.6

        # Early warning signal tracking
        self.variance_history: Dict[str, List[float]] = defaultdict(list)
        self.autocorrelation_history: Dict[str, List[float]] = defaultdict(list)

    def detect_early_warning_signals(
        self,
        agent_id: str,
        state_history: List[float],
        window_size: int = 10
    ) -> Dict[str, float]:
        """
        Detect early warning signals of approaching catastrophe

        Theory: Near critical transitions, systems exhibit:
        1. **Increased variance** (fluctuations grow)
        2. **Increased autocorrelation** (slower recovery from perturbations)
        3. **Critical slowing down** (decay rate → 0)

        These are universal indicators across systems!
        """
        if len(state_history) < window_size:
            return {'variance': 0.0, 'autocorrelation': 0.0, 'warning_level': 0.0}
```

```
    recent = state_history[-window_size:]

    # 1. Increasing variance
    variance = np.var(recent)
    self.variance_history[agent_id].append(variance)

    if len(self.variance_history[agent_id]) >= 3:
        variance_trend = np.polyfit(
            range(len(self.variance_history[agent_id][-10:])),
            self.variance_history[agent_id][-10:],
            1
        )[0]
    else:
        variance_trend = 0.0

    # 2. Autocorrelation at lag-1
    if len(recent) > 1:
        autocorr = np.corrcoef(recent[:-1], recent[1:])[0, 1]
        if np.isnan(autocorr):
            autocorr = 0.0
    else:
        autocorr = 0.0

    self.autocorrelation_history[agent_id].append(autocorr)

    # 3. Combine into warning level
    warning_level = 0.0

    if variance > 2.0:  # High variance
        warning_level += 0.3

    if variance_trend > 0.1:  # Increasing variance
        warning_level += 0.3

    if autocorr > 0.7:  # High autocorrelation (critical slowing)
        warning_level += 0.4

    return {
        'variance': variance,
        'variance_trend': variance_trend,
        'autocorrelation': autocorr,
        'warning_level': min(1.0, warning_level)
    }

def simulate_catastrophe_contagion(
    self,
    triggering_agent: str,
    agent_states: Dict[str, GodhardAwareSystemState]
) -> Dict[str, Tuple[bool, str]]:
    """
    Simulate how catastrophic transition in one agent affects others

    "Catastrophe contagion" - one person's breakdown can trigger others
    """
    contagion_results = {}
```

```
    # Get neighbors of triggering agent
    neighbors = self.network.get_neighbors(triggering_agent)

    for neighbor_id in neighbors:
        if neighbor_id not in agent_states:
            continue

        neighbor_state = agent_states[neighbor_id]

        # Get edge strength
        edge_data = self.network.graph.get_edge_data(triggering_agent, neighbor_id)
        connection_strength = edge_data.get('strength', 1.0) if edge_data else 1.0

        # Contagion probability
        base_prob = self.contagion_strength * connection_strength

        # Higher if neighbor already in critical zone
        if neighbor_state.in_critical_zone:
            prob = base_prob * 1.5
        else:
            prob = base_prob * 0.5

        # Check if contagion occurs
        triggered = np.random.random() < prob

        if triggered:
            contagion_results[neighbor_id] = (
                True,
                f"Triggered by {triggering_agent} via {connection_strength:.2f} connection"
            )
        else:
            contagion_results[neighbor_id] = (False, "Resisted contagion")

    return contagion_results

def compute_collective_hysteresis(
    self,
    group_id: str,
    agent_states: Dict[str, GodhardAwareSystemState]
) -> Dict[str, float]:
    """
    Compute collective hysteresis for a group

    Groups exhibit hysteresis: easier to stress than to recover
    """
    # Get group members
    group_members = [
        agent_id for agent_id, agent in self.network.agents.items()
        if group_id in agent.group_memberships
    ]

    if not group_members:
        return {}

    # Count agents on upper vs lower branch
    on_upper = sum(
        1 for aid in group_members
```

```
            if aid in agent_states and agent_states[aid].on_upper_branch
        )

        on_lower = len(group_members) - on_upper

        # Collective position
        collective_position = np.mean([
            agent_states[aid].curve_position
            for aid in group_members
            if aid in agent_states
        ])

        # Hysteresis width (difference between upper and lower branch means)
        upper_positions = [
            agent_states[aid].curve_position
            for aid in group_members
            if aid in agent_states and agent_states[aid].on_upper_branch
        ]

        lower_positions = [
            agent_states[aid].curve_position
            for aid in group_members
            if aid in agent_states and not agent_states[aid].on_upper_branch
        ]

        if upper_positions and lower_positions:
            hysteresis_width = np.mean(upper_positions) - np.mean(lower_positions)
        else:
            hysteresis_width = 0.0

        return {
            'collective_position': collective_position,
            'fraction_on_upper_branch': on_upper / len(group_members),
            'hysteresis_width': hysteresis_width,
            'in_bistable_region': hysteresis_width > 1.0
        }

    def recommend_anti_catastrophe_interventions(
        self,
        agent_id: str,
        current_state: GodhardAwareSystemState,
        early_warnings: Dict[str, float]
    ) -> List[Dict]:
        """
        Recommend interventions to prevent catastrophic transitions
        """
        interventions = []

        warning_level = early_warnings.get('warning_level', 0.0)

        # Critical zone interventions
        if current_state.in_critical_zone:
            interventions.append({
                'priority': 'CRITICAL',
                'type': 'stability_enhancement',
                'name': 'Increase System Resilience',
                'description': 'Build margin to deepen stability basin and move away from fold',
```

```
            'mechanism': 'Shifts system from bistable to monostable regime',
            'expected_effect': f'+{3.0:.1f} resilience',
            'physics': 'Increases potential well depth'
        })

    # Near fold interventions
    if current_state.distance_to_fold < 2.0:
        interventions.append({
            'priority': 'URGENT',
            'type': 'fold_avoidance',
            'name': 'Emergency Load Reduction',
            'description': 'Immediate pressure relief to move away from catastrophic fold',
            'mechanism': 'Moves state point away from saddle-node bifurcation',
            'expected_effect': f'+{current_state.distance_to_fold:.1f} safety margin',
            'physics': 'Prevents jump to alternative stable state'
        })

    # Early warning signal interventions
    if warning_level > 0.6:
        interventions.append({
            'priority': 'HIGH',
            'type': 'variance_reduction',
            'name': 'Stabilize Fluctuations',
            'description': 'Reduce noise and variability in demands/stressors',
            'mechanism': 'Decreases likelihood of noise-induced transition',
            'expected_effect': f'-{warning_level * 50:.0f}% transition risk',
            'physics': 'Reduces stochastic forcing near bifurcation'
        })

    # Hysteresis escape interventions
    if current_state.on_upper_branch:
        interventions.append({
            'priority': 'MEDIUM',
            'type': 'hysteresis_escape',
            'name': 'Guided Recovery Path',
            'description': 'Sustained moderate reduction to escape high-strain branch',
            'mechanism': 'Must cross middle unstable equilibrium to reach lower branch',
            'expected_effect': 'Jump down to low-strain state',
            'physics': 'Hysteresis requires exceeding return threshold'
        })

    # Resilience building (preventive)
    if current_state.resilience < 5.0:
        interventions.append({
            'priority': 'MEDIUM',
            'type': 'resilience_building',
            'name': 'Deepen Stability Basin',
            'description': 'Build capacity and margin to increase barrier height',
            'mechanism': 'Increases second derivative of potential (curvature)',
            'expected_effect': f'+{5.0 - current_state.resilience:.1f} resilience',
            'physics': 'Steeper potential well = harder to escape'
        })

    return interventions


# ============================================
```

```
# CASCADE DYNAMICS IN GROUPS
# ==========================================

class CascadeDynamics:
    """
    Models cascade effects where one catastrophe triggers others
    Based on network cascade theory
    """

    def __init__(self, network: 'SocialNetwork'):
        self.network = network
        self.cascade_threshold = 0.4  # Fraction of neighbors that trigger cascade

    def simulate_cascade(
        self,
        initial_failures: Set[str],
        agent_vulnerabilities: Dict[str, float],
        max_iterations: int = 100
    ) -> Dict[str, int]:
        """
        Simulate cascade of catastrophic transitions through network

        Returns: {agent_id: iteration_failed}
        """
        failed = set(initial_failures)
        failure_time = {aid: 0 for aid in initial_failures}

        for iteration in range(1, max_iterations + 1):
            new_failures = set()

            for agent_id in self.network.agents.keys():
                if agent_id in failed:
                    continue

                # Count failed neighbors
                neighbors = self.network.get_neighbors(agent_id)
                if not neighbors:
                    continue

                failed_neighbors = sum(1 for n in neighbors if n in failed)
                failure_fraction = failed_neighbors / len(neighbors)

                # Check if threshold exceeded
                vulnerability = agent_vulnerabilities.get(agent_id, 0.5)
                threshold = self.cascade_threshold * (1.0 - vulnerability)

                if failure_fraction >= threshold:
                    new_failures.add(agent_id)
                    failure_time[agent_id] = iteration

            if not new_failures:
                break  # Cascade stopped

            failed.update(new_failures)

        return failure_time
```

```
    def compute_cascade_size_distribution(
        self,
        agent_vulnerabilities: Dict[str, float],
        n_simulations: int = 1000
    ) -> np.ndarray:
        """
        Compute distribution of cascade sizes (power law near critical point)
        """
        cascade_sizes = []

        all_agents = list(self.network.agents.keys())

        for _ in range(n_simulations):
            # Random initial failure
            initial = {np.random.choice(all_agents)}

            # Run cascade
            failures = self.simulate_cascade(initial, agent_vulnerabilities)
            cascade_sizes.append(len(failures))

        return np.array(cascade_sizes)

    def identify_cascade_vulnerable_agents(
        self,
        agent_vulnerabilities: Dict[str, float]
    ) -> List[Tuple[str, float]]:
        """
        Identify agents whose failure would trigger largest cascades
        These are "systemically important" individuals
        """
        vulnerability_scores = []

        for agent_id in self.network.agents.keys():
            # Simulate cascade starting from this agent
            failures = self.simulate_cascade(
                {agent_id},
                agent_vulnerabilities,
                max_iterations=50
            )

            cascade_size = len(failures)
            vulnerability_scores.append((agent_id, cascade_size))

        # Sort by cascade size
        vulnerability_scores.sort(key=lambda x: x[1], reverse=True)

        return vulnerability_scores


# ============================================
#  VISUALIZATION & DIAGNOSTICS
# ============================================

class GodhardVisualization:
    """
    Visualization tools for Godhard curve dynamics
    """
```

```
@staticmethod
def plot_potential_landscape(
    godhard_curve: GodhardCurve,
    pressure: float,
    margin: float,
    current_state: Optional[float] = None
):
    """
    Plot potential energy landscape showing stable/unstable equilibria
    """
    x = np.linspace(-2, 12, 500)
    V = [godhard_curve.potential_function(xi, pressure, margin) for xi in x]
```